

Question 3

File Browser

Search for file name

Actions

Delete forever

Upload

New

Home

 /

user

 /

hadoop

☐	Name	Size	User	Group	Permissions	Date
☐	↑		hdfs	hadoop	drwxr-xr-x	May 08, 2026 10:37 PM
☐	⋮		hadoop	hadoop	drwxrwxrwx	May 08, 2026 11:06 PM
☐	Chicago_Crimes.csv	This folder 65.5 MB	hadoop	hadoop	-rwxr-xr-x	May 08, 2026 11:06 PM

Show 45 of 1 items

Page 1 of 1

⏪

⏩

⏴

⏵

```
• SSH compression : ✓
• SSH-browser      : ✓
• X11-forwarding  : ✗ (disabled or not supported by server)

► For more info, ctrl+click on help or visit our website.

Last login: Sat May 9 06:00:41 2026

 _ | ( _ | /  Amazon Linux AMI
 _ | \ _ | _ |

https://aws.amazon.com/amazon-linux-ami/2018.03-release-notes/
54 package(s) needed for security, out of 80 available
Run "sudo yum update" to apply all updates.

EEEEEEEEEEEEEEEEEEEE MMMMMMMM MMMMMMMM RRRRRRRRRRRRRRRR
E:EEEEEEEEEEEEEEEE M:EEEEEE M:EEEEEE M:EEEEEE R:EEEEEE R:
EE:EEEEEEEEEEEEEEEE M:EEEEEE M:EEEEEE M:EEEEEE R:RRRRRR R:
E:EE EEEEE M:EEEEEE M:EEEEEE M:RRRR R:RR
E:EE M:EEEEEE M:EEEEEE M:EEEEEE R:RR R:RR
E:EEEEEEEEEE M:EEEEEE M:EEEEEE M:EEEEEE R:RRRRRR R:RR
E:EEEEEEEEEE M:EEEEEE M:EEEEEE M:EEEEEE R:RRRRRR R:RR
E:EE M:EEEEEE M:EEEEEE M:EEEEEE R:RR R:RR
E:EE EEEEE M:EEEEEE M:EEEEEE M:EEEEEE R:RR R:RR
EE:EEEEEEEEEEEE M:EEEEEE M:EEEEEE M:EEEEEE R:RR R:RR
E:EEEEEEEEEEEE M:EEEEEE M:EEEEEE M:RRRR R:RR R:RR
EEEEEEEEEEEEEEEE MMMMMMMM MMMMMMMM RRRRRRR RRRRRR

[hadoop@ip-172-31-39-226 ~]$
```

```

FAILED: ParseException line 1:0 cannot recognize input near '/' 'user' '/'
hive> CREATE EXTERNAL TABLE crimes_raw (
  >   crime_id INT,
  >   case_number STRING,
  >   crime_date STRING,
  >   block STRING,
  >   primary_type STRING,
  >   description STRING,
  >   location_desc STRING,
  >   arrest STRING,
  >   domestic STRING,
  >   beat INT,
  >   district INT,
  >   ward INT,
  >   community_area INT,
  >   crime_year INT,
  >   latitude DOUBLE,
  >   longitude DOUBLE
  > )
  > ROW FORMAT DELIMITED
  > FIELDS TERMINATED BY ','
  > STORED AS TEXTFILE
  > TBLPROPERTIES ("skip.header.line.count"="1");
OK

```

```

me taken: 0.468 seconds
hive> LOAD DATA INPATH '/user/hadoop/Chicago_Crimes.csv'
  > INTO TABLE crimes_raw;
Loading data to table default.crimes_raw
me taken: 0.977 seconds

```

```

hive> CREATE TABLE crimes_partitioned (
  >   crime_id INT,
  >   case_number STRING,
  >   crime_date STRING,
  >   block STRING,
  >   primary_type STRING,
  >   description STRING,
  >   location_desc STRING,
  >   arrest STRING,
  >   domestic STRING,
  >   beat INT,
  >   district INT,
  >   ward INT,
  >   community_area INT,
  >   latitude DOUBLE,
  >   longitude DOUBLE
  > )
  > PARTITIONED BY (crime_year INT)
  > CLUSTERED BY (district) INTO 4 BUCKETS
  > ROW FORMAT DELIMITED
  > FIELDS TERMINATED BY ','
  > STORED AS ORC;
me taken: 0.103 seconds
hive> SET hive.exec.dynamic.partition=true;
hive> SET hive.exec.dynamic.partition.mode=nonstrict;

```

```
hive> SET hive.exec.dynamic.partition.mode=nonstrict;
hive> INSERT INTO TABLE crimes_partitioned
> PARTITION (crime_year)
>
> SELECT
>   crime_id,
>   case_number,
>   crime_date,
>   block,
>   primary_type,
>   description,
>   location_desc,
>   arrest,
>   domestic,
>   beat,
>   district,
>   ward,
>   community_area,
>   latitude,
>   longitude,
>   crime_year
> FROM crimes_raw;
Query ID = hadoop_20260509070723_5357036a-1ce2-4417-8157-1dc9436ff721
Total jobs = 1
Launching Job 1 out of 1
Tez session was closed. Reopening...
Session re-established.
Status: Running (Executing on YARN cluster with App id application_1778305107411_0007)
```

	VERTICES	MODE	STATUS	TOTAL	COMPLETED	RUNNING	PENDING	FAILED	KILLED
Map 1	container	SUCCEEDED	1	1	0	0	0	0	0
Reducer 2	container	SUCCEEDED	1	1	0	0	0	0	0

VERTICES: 02/02 [=====>>>] 100% ELAPSED TIME: 18.67 s

```
hive> SELECT COUNT(*) AS total_crimes
> FROM crimes_partitioned;
OK
236756
Time taken: 0.187 seconds, Fetched: 1 row(s)
hive> █
```

```
hive> SELECT primary_type, COUNT(*) AS crime_count
> FROM crimes_partitioned
> GROUP BY primary_type
> ORDER BY crime_count DESC
> LIMIT 5;
Query ID = hadoop_20260509064313_495e9e96-08e8-42d2-a3f3-5be2551ba3d5
Total jobs = 1
Launching Job 1 out of 1
Tez session was closed. Reopening...
Session re-established.
Status: Running (Executing on YARN cluster with App id application_1778305333524_0003)
```

	VERTICES	MODE	STATUS	TOTAL	COMPLETED	RUNNING	PENDING	FAILED	KILLED
Map 1	container	SUCCEEDED	1	1	0	0	0	0	0
Reducer 2	container	SUCCEEDED	1	1	0	0	0	0	0
Reducer 3	container	SUCCEEDED	1	1	0	0	0	0	0

VERTICES: 03/03 [=====>>>] 100% ELAPSED TIME: 8.26 s

```
OK
THEFT      54883
BATTERY    38856
CRIMINAL DAMAGE 27248
MOTOR VEHICLE THEFT      20974
ASSAULT    20485
Time taken: 17.24 seconds, Fetched: 5 row(s)
hive> SELECT crime_year, COUNT(*) AS total_crimes
> FROM crimes_partitioned
> GROUP BY crime_year
> ORDER BY crime_year;
Query ID = hadoop_20260509064338_9bbfe116-d0eb-4fa5-b1da-5d2504ec1f45
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1778305333524_0003)
```

```

-----
VERTICES      MODE      STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
-----
Map 1 ..... container  SUCCEEDED  1        1         0         0         0         0
Reducer 2 ..... container  SUCCEEDED  1        1         0         0         0         0
Reducer 3 ..... container  SUCCEEDED  1        1         0         0         0         0
-----
VERTICES: 03/03 [=====] 100% ELAPSED TIME: 6.61 s
-----
OK
2022    236756
Time taken: 7.476 seconds, Fetched: 1 row(s)
hive>

```

```

Time taken: 7.476 seconds, Fetched: 1 row(s)
hive> SELECT primary_type, COUNT(*) AS arrest_count
> FROM crimes_partitioned
> WHERE LOWER(arrest)='true'
> GROUP BY primary_type
> HAVING COUNT(*) > 500
> ORDER BY arrest_count DESC;
Query ID = hadoop_20260509064455_bbaff7e5-2dc9-47b3-945b-17af1750848a
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1778305333524_0003)

```

```

-----
VERTICES      MODE      STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
-----
Map 1 ..... container  SUCCEEDED  1        1         0         0         0         0
Reducer 2 ..... container  SUCCEEDED  1        1         0         0         0         0
Reducer 3 ..... container  SUCCEEDED  1        1         0         0         0         0
-----
VERTICES: 03/03 [=====] 100% ELAPSED TIME: 7.46 s
-----
OK
WEAPONS VIOLATION      5394
BATTERY 5173
NARCOTICS      4645
OTHER OFFENSE  2094
THEFT      2029
ASSAULT 1900
CRIMINAL TRESPASS      1246
CRIMINAL DAMAGE 899
MOTOR VEHICLE THEFT      583
ROBBERY 558
Time taken: 8.244 seconds, Fetched: 10 row(s)
hive>

```

```

hive> EXPLAIN
> SELECT *
> FROM crimes_partitioned
> WHERE crime_year=2022;
OK
Plan optimized by CBO.

Stage-0
Fetch Operator
  limit:-1
  Select Operator [SEL_2]
    Output:["_col0","_col1","_col2","_col3","_col4","_col5","_col6","_col7","_col8","_col9","_col10","_col11","_col12","_col13","_col14","_col15"]
    TableScan [TS_0]
      Output:["crime_id","case_number","crime_date","block","primary_type","description","location_desc","arrest","domestic","beat","district","ward","community_area","latitude","longitude"]
Time taken: 0.171 seconds, Fetched: 10 row(s)
hive>

```

```
hive> SELECT district, COUNT(*) AS domestic_crime_count
> FROM crimes_partitioned
> WHERE LOWER(domestic)='true'
> GROUP BY district
> ORDER BY domestic_crime_count DESC
> LIMIT 3;
Query ID = hadoop_20260509064715_44605402-a412-4a01-a820-2a9a1d0d0a11
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1778305333524_0003)
```

	VERTICES	MODE	STATUS	TOTAL	COMPLETED	RUNNING	PENDING	FAILED	KILLED
Map 1		container	SUCCEEDED	1	1	0	0	0	0
Reducer 2		container	SUCCEEDED	1	1	0	0	0	0
Reducer 3		container	SUCCEEDED	1	1	0	0	0	0

```
VERTICES: 03/03 [=====] 100% ELAPSED TIME: 7.91 s
OK
6      3821
4      3448
8      3048
Time taken: 8.624 seconds, Fetched: 3 row(s)
hive>
```

QUESTION-5

```
df_airports = df_airports \
    .withColumn(
        "latitude_deg",
        col("latitude_deg").cast(DoubleType())
    ) \
    .withColumn(
        "longitude_deg",
        col("longitude_deg").cast(DoubleType())
    ) \
    .withColumn(
        "elevation_ft",
        col("elevation_ft").cast(IntegerType())
    )
```

> df_airports: pyspark.sql.connect.dataframe.DataFrame = [id: integer, ident: string ... 17 more fields]

```
df_runways = df_runways \
    .withColumn(
        "length_ft",
        col("length_ft").cast(IntegerType())
    ) \
    .withColumn(
        "width_ft",
        col("width_ft").cast(IntegerType())
    )

> df_runways: pyspark.sql.connect.dataframe.DataFrame = [id: integer, airport_ref: integer ... 18 more fields]
```

12:05 PM (<1s) 6 Python

```
print("AIRPORTS SCHEMA")
df_airports.printSchema()
```

root

```
-- id: integer (nullable = true)
-- ident: string (nullable = true)
-- type: string (nullable = true)
-- name: string (nullable = true)
-- latitude_deg: double (nullable = true)
-- longitude_deg: double (nullable = true)
-- elevation_ft: integer (nullable = true)
-- continent: string (nullable = true)
```

```
print("RUNWAYS SCHEMA")
df_runways.printSchema()
```

```
-- id: integer (nullable = true)
-- airport_ref: integer (nullable = true)
-- airport_ident: string (nullable = true)
-- length_ft: integer (nullable = true)
-- width_ft: integer (nullable = true)
-- surface: string (nullable = true)
-- lighted: integer (nullable = true)
-- closed: integer (nullable = true)
-- le_ident: string (nullable = true)
-- le_latitude_deg: double (nullable = true)
-- le_longitude_deg: double (nullable = true)
-- le_elevation_ft: integer (nullable = true)
-- le_heading_degT: double (nullable = true)
-- le_displaced_threshold_ft: integer (nullable = true)
-- he_ident: string (nullable = true)
-- he_latitude_deg: double (nullable = true)
-- he_longitude_deg: double (nullable = true)
-- he_elevation_ft: integer (nullable = true)
-- he_heading_degT: double (nullable = true)
-- he_displaced_threshold_ft: integer (nullable = true)
```

12:06 PM (1s) 8

```
print("Airports Count:", df_airports.count())

print("Runways Count:", df_runways.count())
> See performance \(2\)
```

airports Count: 85324
runways Count: 47884

```
12:10 PM (<1s) 9 Python

df_combined = df_airports.join(
    df_runways,
    df_airports.ident == df_runways.airport_ident,
    "inner"
).select(
    df_airports["name"].alias("airport_name"),
    df_airports["iso_country"],
    df_airports["continent"],
    df_airports["municipality"],
    df_airports["ident"],
    df_runways["airport_ident"],
    df_runways["length_ft"],
    df_runways["width_ft"],
    df_runways["surface"],
    df_runways["lighted"],
    df_runways["closed"]
)
```

```
12:07 PM (<1s) 10 Python

df_combined = df_combined.withColumn(
    "runway_category",
    when(col("length_ft") >= 10000, "Long")

    .when(
        (col("length_ft") >= 5000) &
        (col("length_ft") < 10000),
        "Medium"
    )

    .otherwise("Short")
)

df_combined: pyspark.sql.connect.dataframe.DataFrame = [airport_name: string, iso_country: string ... 10 more fields]
```

```
12:07 PM (1s) 11

df_combined.show(10)
> See performance \(1\)
```

12:07 PM (1s)11

df_combined.show(10)

See performance (1)

airport_name	iso_country	continent	municipality	ident	airport_ident	length_ft	width_ft	surface	lighted	closed	runway_category
Total RF Heliport	US	NA	Bensalem	00A	00A	80	80	ASPH-G	1	0	Short
Lowell Field	US	NA	Anchor Point	00AK	00AK	2500	40	GRVL	0	0	Short
Epps Airpark	US	NA	Harvest	00AL	00AL	2100	90	TURF	0	0	Short
Katmai Lodge Airport	US	NA	King Salmon	00AN	00AN	4517	60	GVL	0	0	Short
Fulton Airport	US	NA	Alex	00AS	00AS	1450	60	Turf	0		
Cordes Airport	US	NA	Cordes	00AZ	00AZ	1700	60	GRAVEL	0		
Goldstone (GTS) A...	US	NA	Barstow	00CA	00CA	6000	80	ASPH	0		
Grass Patch Airport	US	NA	Bushnell	00FA	00FA	3200	100	TURF	0		
River Oak Airport	US	NA	Okeechobee	00FL	00FL	4700	60	TURF	0		
Lt World Airport	US	NA	Lithonia	00GA	00GA	2600	80	TURF	0		

only showing top 10 rows

12

df_combined.show(10)

See performance (1)

```
df_combined.createOrReplaceTempView("airports_view")
```

Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.

12:11 PM (<1s)14

df_combined.createOrReplaceTempView("airports_view")

See performance (1)

12:11 PM (5s)15

%sql

See performance (1)

Optimize

```
SELECT iso_country,
       COUNT(*) AS runway_count
FROM airports_view
GROUP BY iso_country
ORDER BY runway_count DESC
LIMIT 10;
```

Table

	iso_country	runway_count
1	US	26450
2	BR	5825
3	AU	2103
4	CA	1578
5	AR	761

12:12 PM (1s)16SQL

```
%sql

SELECT continent,
       AVG(length_ft) AS avg_runway_length
FROM airports_view
WHERE length_ft IS NOT NULL
GROUP BY continent
ORDER BY avg_runway_length DESC;
```

See performance (1)Optimize

Table +

	continent	1.2 avg_runway_length
1	AN	7768.857142857143
2	AS	6925.6469996647675
3	AF	6787.526448362721
4	EU	4103.151335311572
5	OC	3558.0401785714284
6	SA	2857.0891141239963
7	NA	2599.724218941758

7 rows | 1.30s runtimeRefreshed 48 min

12:13 PM (2s)17SQL

```
%sql

SELECT airport_name,
       municipality,
       COUNT(*) AS lighted_runways
FROM airports_view
WHERE iso_country = 'IN'
AND lighted = 1
GROUP BY airport_name, municipality
ORDER BY lighted_runways DESC;
```

See performance (1)Optimize

Table +

	airport_name	municipi...	lighted_runways
1	Indira Gandhi International Airport	New Delhi	3
2	Navi Mumbai International Airport	Navi Mumbai	2
3	Ambala Air Force Station	Ambala	2
4	Agra Airport / Agra Air Force Station	Agra	2
5	Raja Bhoj International Airport	Bhopal	2
6	Chhatrapati Shivaji Maharaj International Airport	Mumbai	2
7	Prayagraj Airport	Allahabad	2
8	Netaji Subhash Chandra Bose International Airp...	Kolkata	2

%sql

SELECT airport name,
iso_country,
MAX(length ft) AS max_runway_length
FROM airports view
GROUP BY airport name, iso_country
ORDER BY max_runway_length DESC
LIMIT 5;

> [See performance \(1\)](#) [Optimize](#)

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [airport_name: string, iso_country: string ... 1 more field]

Table +

Q F

	airport_name	iso_country	max_runway_length
1	Gunflint Seaplane Base	US	30000
2	Libby Camps Seaplane Base	US	26000
3	Brookville Reservoir Seaplane Ba...	US	25000
4	Long Lake Seaplane Base	US	25000
5	Conchas Lake Seaplane Base	US	21120

5 rows | 1.74s runtime

Refreshed 48 minut

Question-4

```
use ecommerce
switched to db ecommerce
db.createCollection("orders")
{ ok: 1 }
db.orders.insertMany([
{
  "order_id": "ORD001",
  "customer": "Alice",
  "city": "Mumbai",
  "category": "Electronics",
  "product": "Laptop",
  "quantity": 1,
  "price": 75000,
  "status": "delivered",
  "date": "2024-01-15"
},
{
  "order_id": "ORD002",
  "customer": "Bob",
  "city": "Delhi",
  "category": "Clothing",
  "product": "Jacket",
  "quantity": 3,
  "price": 4500,
  "status": "pending",
  "date": "2024-02-20"
}
```

```

acknowledged: true,
insertedIds: {
  '0': ObjectId('69fed326169c96bcaff93b51'),
  '1': ObjectId('69fed326169c96bcaff93b52'),
  '2': ObjectId('69fed326169c96bcaff93b53'),
  '3': ObjectId('69fed326169c96bcaff93b54'),
  '4': ObjectId('69fed326169c96bcaff93b55')
}

p.orders.find(

{ status: "delivered" }

, {
  _id: ObjectId('69fed326169c96bcaff93b51'),
  order_id: 'ORD001',
  customer: 'Alice',
  city: 'Mumbai',
  category: 'Electronics',
  product: 'Laptop',
  quantity: 1,
  price: 75000,
  status: 'delivered',
  date: '2024-01-15'
}

```

```

db.orders.updateOne(

{ order_id: "ORD002" },

{ $set: { status: "shipped" } }

)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
db.orders.find(

{ order_id: "ORD002" }

)
{
  _id: ObjectId('69fed326169c96bcaff93b52'),
  order_id: 'ORD002',
  customer: 'Bob',
  city: 'Delhi',
  category: 'Clothing',
  product: 'Jacket',
  quantity: 3,
  price: 4500
}

```

```
> db.orders.insertOne({
  "order_id": "ORD006",
  "customer": "Frank",
  "city": "Bangalore",
  "category": "Electronics",
  "product": "Smart Watch",
  "quantity": 2,
  "price": 15000,
  "status": "pending",
  "date": "2024-05-10"
})
< {
  acknowledged: true,
```

```
> db.orders.deleteOne(
  { order_id: "ORD004" }
)
< {
  acknowledged: true,
  deletedCount: 1
}
> db.orders.find(
  { order_id: "ORD004" }
)
<
> db.orders.aggregate([
  {
    $group: {
      _id: "$category",
      total_revenue: {
        $sum: {
```

```

db.orders.aggregate([
  {
    $group: {
      _id: "$city",
      total_orders: { $sum: 1 },
      average_price: { $avg: "$price" }
    }
  }
])
{
  _id: 'Mumbai',
  total_orders: 2,
  average_price: 50000
}

```

```

db.orders.aggregate([
  {
    $group: {
      _id: "$status",
      total_orders: { $sum: 1 }
    }
  },
  {
    $match: {
      total_orders: { $gt: 1 }
    }
  }
])

```